Bank Transaction Analytics and Fraud Detection

Technical Documentation

Design decisions, architecture, and a walkthrough of every file

Submitted by: _____

Roll number: _____

Course / Instructor: _____

Live dashboard: https://jayquery-007-bank-transaction-analytics.static.hf.space
Source code: https://github.com/JayQuery-007/bank_transaction_analysis
October 2026

Contents

1. Project at a glance 4
1.1 How to read this document 4
2. Why this project 5
2.1 Why bank transaction fraud 5
2.2 Why the PaySim dataset 5
3. Why these technologies and design choices 6
3.1 Technology choices 6
3.2 Design decisions 6
4. Architecture 8
4.1 Two ways the dashboard runs 9
5. How the system works, step by step 10
5.1 From a CSV row to the dashboard 10
5.2 What happens when the dashboard loads 10
5.3 What happens when you click "Score transaction" 10
6. Repository map 12
6.1 Generated locally, not in Git 13
7. File-by-file walkthrough 14
7.1 Configuration and entry points 14
7.2 Data download and shared Spark code 15
7.3 Pipeline jobs 16
7.4 Windows file system support (spark_jobs/winfs) 19
7.5 Exploratory analysis notebook 20
7.6 Backend API (backend/app) 21
7.7 Dashboard (frontend) 22
7.8 Deployment files 24
7.9 Pipeline outputs committed to the repository 26
8. How the model works 28
8.1 Where the labels come from 28
8.2 Features 28
8.3 Decision trees and the forest 28
8.4 From votes to a decision 28
8.5 Evaluation 29
9. Data formats 30
9.1 Parquet lake schema (data/lake/transactions) 30
9.2 MongoDB 30
10. API reference (server mode) 31
11. Deployment 32
11.1 Why a static site 32
11.2 How to redeploy 32

11.3 Server deployment alternative 32
12. Running and reproducing 33
13. Problems encountered and how they were solved 34
14. Testing and verification 35
15. Limitations and future work 36
Glossary 36

1. Project at a glance

This project is a batch big data pipeline for bank transaction analytics and fraud detection. It processes 6,362,620 mobile-money transactions (494 MB) with Apache Spark, finds where, when and how fraud happens using Spark SQL, trains a fraud detection model with Spark MLlib, stores the results in MongoDB, and presents them on a web dashboard.

Item	Value
Dataset	PaySim (Kaggle: ealaxi/paysim1), 6,362,620 transactions over 31 days
Course technology combination	Python + Spark + MongoDB + Machine Learning
Other technologies	Parquet, FastAPI, Chart.js, Jupyter, Docker, Hugging Face Spaces
Fraud cases	8,197 (0.129% of transactions)
Model	Random Forest, 60 trees, trained on 1.94M transactions with Spark MLlib
Model result (unseen test set)	2,401 of 2,407 frauds caught, 17 false alarms
Existing rule result (same test set)	4 of 2,407 frauds caught
Live dashboard	https://jayquery-007-bank-transaction-analytics.static.hf.space
Source code	https://github.com/JayQuery-007/bank_transaction_analysis
Hand-written code	about 1,580 lines across Python, Java, JavaScript, HTML and CSS

1.1 How to read this document

- **Sections 2–3** explain why this project and why each technology and design choice was made, including the alternatives that were rejected.
- **Sections 4–5** describe the architecture and how data moves through the system, from a raw CSV row to a number on the dashboard.
- **Section 6** lists every file in the repository. **Section 7** walks through each file: what it does, how it works, and the key code.
- **Sections 8–10** cover the model's internals, the data formats and the API.
- **Sections 11–15** cover deployment, running the project, the problems that came up, testing, and limitations.

2. Why this project

2.1 Why bank transaction fraud

- **It is a real, high-value problem.** Payment fraud is a direct financial loss for banks and customers, and every bank runs a fraud detection system.
- **It is naturally a big data problem.** Transaction logs are large, keep growing every day, and have to be analysed in full. Sampling can miss rare events, and fraud is exactly such a rare event.
- **It needs no hardware.** Several course options require ESP32, Arduino or sensors. This project uses only software and a public dataset.
- **Success is measurable.** Because the data has fraud labels, the project can prove how well it works with real numbers (frauds caught, false alarms) instead of only showing charts.
- **It combines analytics and machine learning.** The same data supports descriptive analytics (Spark SQL), predictive modelling (MLlib) and serving (MongoDB, API), which covers most of the course syllabus in one project.

2.2 Why the PaySim dataset

Real bank transaction data with fraud labels is never published, because of customer privacy and banking regulations. The options considered:

Dataset	Size	Why it was or wasn't chosen
PaySim (chosen)	6.36M rows, 494 MB	Large enough to justify Spark. Readable columns (type, amount, balances) that support real analytics. Fraud labels included. Simulated from one month of real logs of a mobile money provider. The standard public benchmark for this problem.
Credit Card Fraud (ULB)	284,807 rows	Real transactions, but the features are anonymised (V1–V28 from PCA), so there is nothing meaningful to analyse beyond the amount. Too small for a big data course.
IEEE-CIS Fraud Detection	590,540 rows, 400+ columns	Competition dataset with complex, mostly anonymised columns. Harder to explain and smaller than PaySim.
Berka (Czech bank, 1999)	1M transactions	Real bank data, but it has no fraud labels, so there is no way to measure a fraud detector.
Synthetic generator	any size	Rejected. An earlier version of this project generated its own data, but self-made data proves nothing, because the model just finds the patterns that were programmed in.

> **Being honest about PaySim:** it is a published simulation based on real logs, not raw bank records. That is the trade-off for having labelled data, and Section 15 discusses how it affects the model's scores.

3. Why these technologies and design choices

3.1 Technology choices

Need	Chosen	Alternatives considered	Reason
Processing engine	**Apache Spark**	pandas; Hadoop MapReduce; Hive	pandas loads everything into one machine's memory and does not scale out. MapReduce is slow and verbose: every step writes to disk and needs hand-written Java. Spark does the same distributed work in memory, with SQL and ML built in, and the same code runs on a laptop or a cluster.
Analytics	**Spark SQL**	Hive; DataFrame API only	Plain SQL is easy to read and review. Spark's Catalyst optimiser plans the query, and no separate Hive metastore is needed.
Storage format	**Parquet**	CSV; JSON; Avro	Columnar and compressed: 494 MB of CSV became 315 MB with six extra columns. It stores the schema and supports partition pruning. CSV has to be re-parsed on every read.
Machine learning	**Spark MLlib**	scikit-learn	MLlib trains on all 1.94M training rows in parallel, directly on the Spark DataFrame. scikit-learn would need the data copied into one process's memory, and possibly sampled.
Serving database	**MongoDB**	MySQL; PostgreSQL	Results are naturally JSON documents of different shapes (13 result sets). MongoDB stores each without designing 13 table schemas, and documents map directly to API responses.
API	**FastAPI**	Flask; Django	Small and fast. It validates input with type hints (Pydantic), which gives free validation for the prediction form, and it serves the static frontend too.
Dashboard	**HTML + Chart.js**	Power BI; Tableau; Grafana; React	Runs in any browser with no licence or install, can be hosted for free, and has no build step. Chart.js is bundled so it works offline.
Hosting	**Hugging Face static Space**	Docker Space; Render; GitHub Pages	Free and permanent. Docker Spaces and servers with enough memory for Spark cost money, so the model was exported to run in the browser (Section 11).

3.2 Design decisions

Batch layer + serving layer

The 6.36M rows stay in the Parquet lake, where Spark processes them. MongoDB holds only the small results the dashboard needs (a few thousand documents). Copying millions of rows into the serving database would make it slower and bigger without any benefit to the user. This split is the standard pattern in big data systems.

Partition the lake by transaction type

Every analysis and the model filter by type, and fraud exists only in TRANSFER and CASH_OUT. Partitioning by type means Spark reads only the two folders it needs (2.77M rows) and skips 3.6M rows without opening them.

Explicit schema instead of inferSchema

Schema inference reads the whole 494 MB file once just to guess the types, then reads it again. An explicit schema avoids that extra pass and prevents wrong guesses.

Supervised model, compared against baselines

The data has labels, so a supervised model can be trained and, more importantly, measured. Two baselines show the model is worth it: the provider's existing rule (isFlaggedFraud), and a simple linear model (Logistic Regression). Random Forest was selected because the fraud signal depends on combinations of conditions, such as "balance went to zero and the amount equals the old balance", which trees capture and linear models do not.

Measure with precision, recall and AUC-PR, not accuracy

Fraud is 0.3% of the training data. A model that never predicts fraud is 99.7% accurate and useless, so accuracy is never reported.

Commit the results and model, not the data

The raw data and the lake are hundreds of MB and are regenerated by the pipeline, so they are not committed. The results (under 1 MB) and the model (347 KB) are committed, so anyone can open the dashboard right after cloning, without Java, MongoDB or a 494 MB download.

Keep all Spark work on the JVM side

No Python UDFs or RDD operations are used, only DataFrames, SQL and MLlib. This is faster, because no data is copied between the JVM and Python. It also avoids a crash of PySpark's Python workers on Windows with Python 3.12+ (Section 13).

4. Architecture

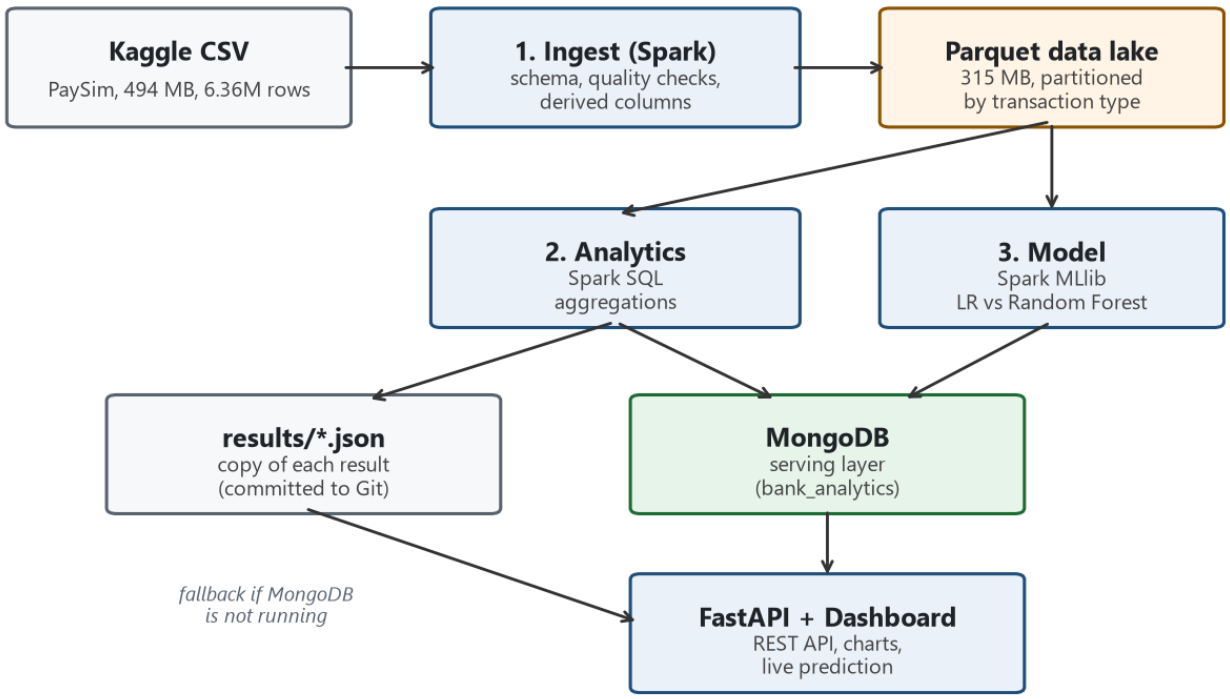


Figure 1. Data flow: CSV → Spark → Parquet lake → analytics and model → MongoDB → API and dashboard

Layer	Components	Responsibility
Ingestion	data/download_data.py, spark_jobs/ingest.py	Get the raw data, check quality, clean, add features, write the lake
Storage (batch layer)	data/lake/transactions/ (Parquet)	All 6.36M cleaned rows, partitioned by type
Processing	spark_jobs/analytics.py, spark_jobs/train_model.py	Spark SQL aggregations; MLlib training and evaluation
Serving layer	MongoDB (bank_analytics), results/*.json	Small result sets ready for display
Model store	models/fraud_rf/	Trained Random Forest in Spark ML format
Application	backend/app/ (FastAPI)	REST API and live prediction
Presentation	frontend/	Dashboard
Deployment	deploy/, Dockerfile	Static site on Hugging Face; container image for other hosts

4.1 Two ways the dashboard runs

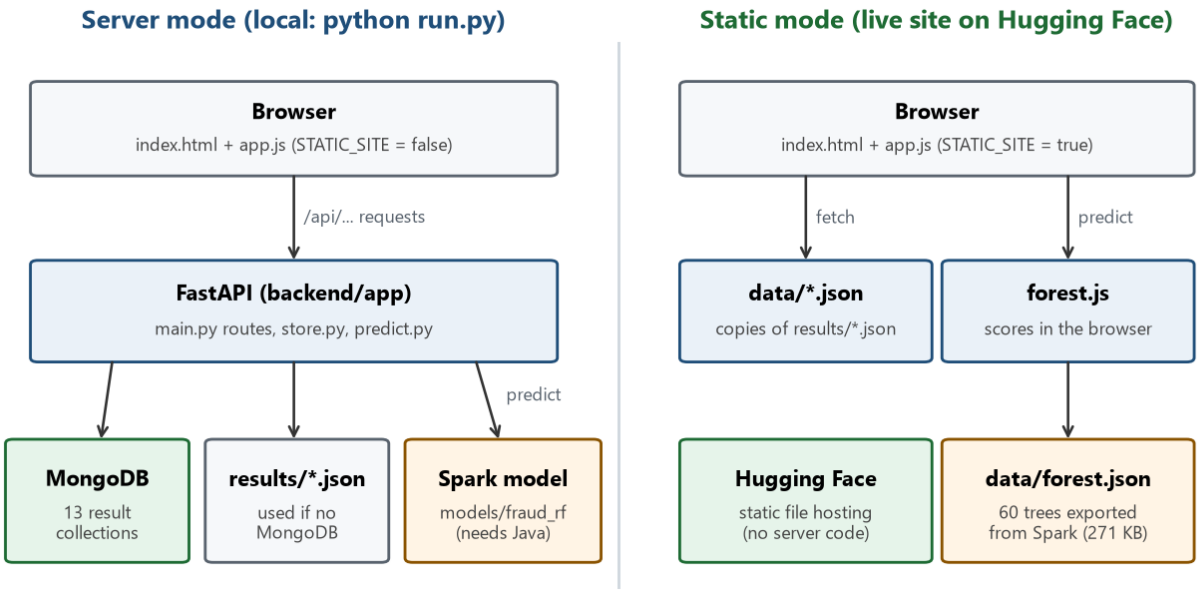


Figure 2. Server mode (local) and static mode (live site)

- **Server mode** `python run.py`: the browser calls the FastAPI backend, which reads MongoDB (or the results files if MongoDB isn't running). Predictions use the saved Spark model, so Java is required.
- **Static mode** (the live site): there is no server. The browser loads the same results as plain JSON files, and predictions use the Random Forest exported to `forest.json`, evaluated by `forest.js`. The export was checked to give exactly the same probabilities as Spark.

The same `index.html` and `app.js` are used in both modes. One flag in `config.js` (`window.STATIC_SITE`) decides which data source the page uses.

5. How the system works, step by step

5.1 From a CSV row to the dashboard

Follow one transaction through the pipeline:

1. **Raw CSV.** A line such as `1,TRANSFER,181.0,C1305486145,181.0,0.0,C553264065,0.0,0.0,1,0` in the 494 MB file.
2. **Ingest.** Spark reads it with the explicit schema, renames columns, and adds derived columns: day = 1, hour_of_day = 1, error_balance_orig = 0 + 181 − 181 = 0, error_balance_dest = 0 + 181 − 0 = 181, orig_emptied = 1. The row is written to `data/lake/transactions/type=TRANSFER/`.
3. **Analytics.** Spark SQL counts it in the TRANSFER row of type_summary, in hour 1 of hourly, in the 100–1k band of amount_bands, and in the fraud row of balance_patterns. Each result set is written to MongoDB and results/.
4. **Model.** The row is either in the training set (the model learns from it) or in the test set (the model is scored on it). If it is in the test set and flagged, it is saved in flagged_transactions with its probability and outcome.
5. **Serving.** The dashboard fetches these result sets and draws them. The transaction itself is never sent to the browser. Only aggregates and the flagged test transactions are.

5.2 What happens when the dashboard loads

1. The browser loads `index.html`, then `chart.umd.min.js`, `config.js`, `forest.js` and `app.js`.
2. `app.js` picks the data layer based on `window.STATIC_SITE`.
3. `main()` requests 11 result sets in parallel (`Promise.all`) plus the data source label.
4. Each render function fills one section: KPI cards, tables, and Chart.js charts. The finding sentences are built from the data, for example the peak fraud hour.
5. The flagged-transactions table loads page 1 (25 rows), and the prediction form is pre-filled with a normal cash-out.

5.3 What happens when you click "Score transaction"

Example: a CASH_OUT of 1,096,187.24 at 2:00 where the sender's balance goes from 1,096,187.24 to 0.00.

Server mode

1. `app.js` sends `POST /api/predict` with the 7 form fields as JSON.
2. FastAPI validates the input with the `Transaction` Pydantic model: the type must be TRANSFER or CASH_OUT, amounts must be ≥ 0, and the hour must be 0–23. Invalid input returns HTTP 422.
3. `predict.py` starts Spark on the first call (5–30 s) and loads the saved PipelineModel. Later calls reuse both.
4. It computes the 9 features, creates a one-row DataFrame, runs `model.transform()` and reads the fraud probability from the probability vector.
5. The response contains the probability, the fraud/legit decision, and both balance errors.

Static mode

1. `app.js` calls `scoreWithForest()` in `forest.js`, with no network call except fetching `forest.json` the first time.
2. The same 9 features are computed, and the transaction is routed through all 60 trees (go left if feature ≤ threshold).

3. Each tree's leaf gives a fraud probability. The average across the trees is the answer, and it is fraud if above 0.5. This takes a few milliseconds.

6. Repository map

Every file in the repository, grouped by folder. Section 7 explains each one in detail.

Path	Purpose
`README.md`	Project overview, results, how to run and deploy
`config.py`	Shared paths and settings (data, lake, model, results, MongoDB)
`run_pipeline.py`	Runs the four pipeline steps in order
`run.py`	Starts the dashboard server (FastAPI via Uvicorn)
`start.ps1`	Windows helper: starts the local MongoDB, then the dashboard
`requirements.txt`, `requirements-app.txt`	Python dependencies (full set; runtime-only set for the container)
`Dockerfile`, `.dockerignore`	Container image for server-based hosting
`.gitignore`, `.gitattributes`	What Git ignores; which files Git treats as binary
`data/download_data.py`	Downloads PaySim from Kaggle
`spark_jobs/__init__.py`	Marks spark_jobs as a Python package (empty)
`spark_jobs/common.py`	Spark session setup; publishing results to MongoDB and results/
`spark_jobs/ingest.py`	Pipeline step 1: CSV → Parquet lake
`spark_jobs/analytics.py`	Pipeline step 2: Spark SQL analytics
`spark_jobs/train_model.py`	Pipeline step 3: MLlib model training and evaluation
`spark_jobs/winfs/src/bda/*.java`	Hadoop file system classes that let Spark write files on Windows
`spark_jobs/winfs/winfs.jar`	Compiled version of the two Java classes (2 KB)
`notebooks/eda.ipynb`	Exploratory data analysis with plots
`backend/app/__init__.py`	Marks backend/app as a package (empty)
`backend/app/main.py`	FastAPI routes and input validation
`backend/app/store.py`	Reads results from MongoDB or results/*.json
`backend/app/predict.py`	Scores one transaction with the saved Spark model
`frontend/index.html`	Dashboard page structure (7 sections)
`frontend/style.css`	Dashboard styling
`frontend/app.js`	Dashboard logic: data loading, charts, tables, form
`frontend/forest.js`	Random Forest scoring in the browser (static mode)
`frontend/config.js`	Mode flag: server (false) or static (true)
`frontend/vendor/chart.umd.min.js`	Chart.js 4.4.4 charting library (bundled)
`deploy/build_static.py`	Builds the static site and exports the model to JSON
`deploy/deploy_hf.py`	Uploads the static site to Hugging Face
`deploy/space_README.md`	README shown on the Hugging Face Space page
`models/fraud_rf/`	Trained Random Forest pipeline (Spark ML format, 347 KB)
`results/*.json`	13 result sets produced by the pipeline
`docs/`	Project report (Word and PDF) and this document

6.1 Generated locally, not in Git

Path	Created by	Why not committed
`data/raw/paysim.csv`	download_data.py	494 MB, over GitHub's file size limit; downloadable from Kaggle
`data/lake/transactions/`	ingest.py	315 MB; rebuilt from the CSV in about 40 s
`site/`	build_static.py	Build output, regenerated on every deploy
`venv/`	python -m venv	Each machine creates its own
`jdk/`, `mongodb/`	manual setup on the development laptop	Portable Java and MongoDB copies, hundreds of MB, platform-specific

7. File-by-file walkthrough

7.1 Configuration and entry points

config.py

Single place for all paths and settings · 10 lines

Every other module imports its paths from here, so nothing has hard-coded locations. `ROOT` is computed from the file's own location, so the project works wherever it is cloned. `MONGO_URI` can be overridden with an environment variable, for example to point at a cloud MongoDB.

```
import os
from pathlib import Path

ROOT = Path(__file__).resolve().parent

RAW_CSV = ROOT / "data" / "raw" / "paysim.csv"
LAKE_DIR = ROOT / "data" / "lake" / "transactions"
MODEL_DIR = ROOT / "models" / "fraud_rf"
RESULTS_DIR = ROOT / "results"

KAGGLE_DATASET = "ealaxi/paysim1"

MONGO_URI = os.environ.get("MONGO_URI", "mongodb://localhost:27017")
MONGO_DB = "bank_analytics"
```

config.py, complete file

run_pipeline.py

Runs the whole batch pipeline · 19 lines

Runs the four steps as separate Python processes, in order, and stops at the first failure. Each step runs as a module (`python -m spark_jobs.ingest`) so that imports like `from config import ...` resolve from the project root. `sys.executable` makes sure the same Python (the virtual environment) is used for every step.

```
STEPS = [
    ("Download PaySim dataset from Kaggle", [sys.executable, "data/download_data.py"]),
    ("Ingest CSV into Parquet data lake", [sys.executable, "-m", "spark_jobs.ingest"]),
    ("Spark SQL analytics -> MongoDB", [sys.executable, "-m", "spark_jobs.analytics"]),
    ("Train and evaluate fraud model (Spark MLlib)", [sys.executable, "-m", "spark_jobs.train_model"]),
]
```

run_pipeline.py, lines 9–14

run.py

Starts the dashboard server · 17 lines

Changes into the project folder, adds it to the import path, and starts Uvicorn (the web server) with the FastAPI app. The host and port come from environment variables. Locally it listens on 127.0.0.1:8000, which only this computer can reach. The Dockerfile sets HOST=0.0.0.0 and PORT=7860 for hosting.

```
if __name__ == "__main__":
    # HOST/PORT are set by the Dockerfile when deployed; locally it stays on localhost:8000
    host = os.environ.get("HOST", "127.0.0.1")
    port = int(os.environ.get("PORT", "8000"))
    print(f"Dashboard: http://localhost:{port}")
    uvicorn.run("backend.app.main:app", host=host, port=port)
```

run.py, lines 16–21

start.ps1

Windows convenience script · 11 lines

If a portable MongoDB exists in `mongodb/` and nothing is already listening on port 27017, it starts `mongod` hidden in the background with the project's data and log folders. Then it runs `run.py` with the virtual environment's Python.

7.2 Data download and shared Spark code

data/download_data.py

Downloads the dataset · 24 lines

Skips the download if `data/raw/paysim.csv` already exists. Otherwise it uses `kagglehub` (Kaggle's official client, which reads the user's Kaggle token from `~/.kaggle/`) to download the dataset into its cache, then copies the CSV into the project. If there's no token, the CSV can be downloaded from the Kaggle website by hand.

```
def main():
    if RAW_CSV.exists():
        print(f"{RAW_CSV} already exists ({RAW_CSV.stat().st_size / 1e6:.0f} MB), skipping download")
        return

    import kagglehub

    folder = Path(kagglehub.dataset_download(KAGGLE_DATASET))
    csv_files = list(folder.glob("*.csv"))
    if not csv_files:
        sys.exit(f"No CSV found in {folder}")

    RAW_CSV.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy(csv_files[0], RAW_CSV)
    print(f"Saved {RAW_CSV} ({RAW_CSV.stat().st_size / 1e6:.0f} MB)")
```

data/download_data.py, lines 15–29

spark_jobs/common.py

Spark session setup and result publishing, used by every job and the API · 64 lines

get_spark(app_name)

Creates the SparkSession with the project's settings:

- `_find_java()` makes sure Java is available: JAVA_HOME, then the PATH, then a JDK bundled in the project's `jdk/` folder. Otherwise it exits with a clear message.
- `PYSPARK_PYTHON` is set to the current interpreter, so Spark never picks up a different Python.
- `local[*]` runs Spark on all CPU cores of one machine. Driver memory defaults to 4 GB (2 GB in the container).
- `spark.sql.shuffle.partitions = 16` replaces the default of 200, which is far too many for one machine and makes small aggregations slow.
- Arrow is enabled for fast conversion of results to pandas.
- On Windows only, the custom file system from `winfs.jar` is plugged in (Section 7.4).

```
def get_spark(app_name):
    _find_java()
    os.environ["PYSPARK_PYTHON"] = sys.executable

    from pyspark.sql import SparkSession

    builder = (
        SparkSession.builder.appName(app_name)
        .master("local[*]")
        .config("spark.driver.memory", os.environ.get("SPARK_DRIVER_MEMORY", "4g"))
        .config("spark.sql.shuffle.partitions", "16")
        .config("spark.sql.execution.arrow.pyspark.enabled", "true")
        .config("spark.ui.showConsoleProgress", "false")
    )
    if os.name == "nt":
        # see spark_jobs/winfs/src - avoids needing winutils.exe/hadoop.dll on Windows
        builder = (
            builder.config("spark.driver.extraClassPath", str(WINFS_JAR))
            .config("spark.hadoop.fs.file.impl", "bda.WinLocalFileSystem")
        )

    spark = builder.getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")
    return spark
```

spark_jobs/common.py, lines 22–45

publish(name, records)

Every result set goes through this function. It converts values to plain JSON types (`_clean` handles dates and numpy numbers), writes `results/<name>.json`, then replaces the contents of the MongoDB collection with the same name. If MongoDB isn't reachable within 2 seconds, it prints a note and carries on, so the pipeline also works without a database.

```
def publish(name, records):
    """Write a result set to MongoDB (serving layer) and to results/<name>.json."""
    if isinstance(records, dict):
        records = [records]
    records = _clean(records)

    RESULTS_DIR.mkdir(exist_ok=True)
    with open(RESULTS_DIR / f"{name}.json", "w") as f:
        json.dump(records, f, indent=1)

    try:
        from pymongo import MongoClient

        client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=2000)
        coll = client[MONGO_DB][name]
        coll.delete_many({})
        if records:
            coll.insert_many([dict(r) for r in records])
        print(f"  -> {name}: {len(records)} docs written to MongoDB and results/{name}.json")
    except Exception as e:
        print(f"  -> {name}: written to results/{name}.json (MongoDB unavailable: {type(e).__name__})")
```

spark_jobs/common.py, lines 60–80

to_records(df)

Converts a (small) Spark result DataFrame to a list of dictionaries through pandas, using Arrow. Only aggregated results go through it, never the 6M-row data.

7.3 Pipeline jobs

spark_jobs/ingest.py

Step 1: raw CSV → cleaned, enriched Parquet lake · 85 lines

1. Read with an explicit schema

```
RAW_SCHEMA = StructType([
    StructField("step", IntegerType()),
    StructField("type", StringType()),
    StructField("amount", DoubleType()),
    StructField("nameOrig", StringType()),
    StructField("oldbalanceOrg", DoubleType()),
    StructField("newbalanceOrig", DoubleType()),
    StructField("nameDest", StringType()),
    StructField("oldbalanceDest", DoubleType()),
    StructField("newbalanceDest", DoubleType()),
    StructField("isFraud", IntegerType()),
    StructField("isFlaggedFraud", IntegerType()),
])
```

spark_jobs/ingest.py, lines 18–30

2. Rename and cache

Columns are renamed to snake_case (`oldbalanceOrg` → `old_balance_orig`, which also fixes the source's inconsistent "Org"/"Orig" spelling). The DataFrame is cached, because the quality checks scan it several times.

3. Data quality checks

Null counts for every column (computed in one pass), negative and zero amounts, duplicate rows (row count minus distinct rows), and distinct sender and receiver accounts.

```
# data quality checks
total = df.count()
```

```
null_counts = df.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in
df.columns]).first().asDict()
quality = {
    "total_rows": total,
    "rows_with_nulls": int(sum(v or 0 for v in null_counts.values())),
    "negative_amounts": df.filter(F.col("amount") < 0).count(),
    "zero_amounts": df.filter(F.col("amount") == 0).count(),
    "duplicate_rows": total - df.dropDuplicates().count(),
    "distinct_origin_accounts": df.select("name_orig").distinct().count(),
    "distinct_destination_accounts": df.select("name_dest").distinct().count(),
}
print("Data quality:", quality)
```

spark_jobs/ingest.py, lines 56–68

4. Clean and derive columns

```
# step = 1 hour of simulated time, 744 steps = 31 days
df = (
    df.filter(F.col("amount") > 0)
    .withColumn("day", ((F.col("step") - 1) / 24).cast("int") + 1)
    .withColumn("hour_of_day", F.col("step") % 24)
    .withColumn("dest_is_merchant", F.col("name_dest").startswith("M").cast("int"))
    # how far the recorded balances are from what the amount says they should be
    .withColumn("error_balance_orig", F.col("new_balance_orig") + F.col("amount") -
F.col("old_balance_orig"))
    .withColumn("error_balance_dest", F.col("old_balance_dest") + F.col("amount") -
F.col("new_balance_dest"))
    .withColumn(
        "orig_emptied",
        ((F.col("old_balance_orig") > 0) & (F.col("new_balance_orig") == 0)).cast("int"),
    )
)
```

spark_jobs/ingest.py, lines 70–83

5. Write the partitioned Parquet lake

`repartition("type")` groups each type's rows together first, so each partition folder gets a few large files instead of many small ones. `partitionBy("type")` then creates one folder per type. Finally the job reads the lake back to confirm the row count and publishes the `data_quality` result.

```
df.repartition("type")
.write.mode("overwrite")
.partitionBy("type")
.parquet(str(LAKE_DIR))
```

spark_jobs/ingest.py, lines 86–89

spark_jobs/analytics.py

Step 2: descriptive analytics in Spark SQL · 104 lines

The queries are kept in a dictionary, `QUERIES`, mapping each result name to its SQL text. `main()` registers the lake as a SQL view called `tx`, caches it in memory (so the lake is read from disk once, not seven times), then runs each query and publishes the result.

```
def main():
    spark = get_spark("paysim-analytics")
    spark.read.parquet(str(LAKE_DIR)).createOrReplaceTempView("tx")
    spark.catalog.cacheTable("tx")

    for name, sql in QUERIES.items():
        publish(name, to_records(spark.sql(sql)))

    spark.stop()
```

spark_jobs/analytics.py, lines 107–115

Query	What it computes
`overview`	One row of totals: transactions, value, fraud count and value, fraud rate, flagged by the existing rule, distinct accounts, days.
`type_summary`	GROUP BY type: count, total and average amount, fraud count and rate.
`hourly`	GROUP BY hour_of_day: volume, fraud count, fraud rate.
`daily`	GROUP BY day: volume, fraud count, total amount.
`amount_bands`	TRANSFER/CASH_OUT only. Bands by order of magnitude, FLOOR(LOG10(amount)) clamped to 1–7, with legit and fraud counts per band.
`balance_patterns`	TRANSFER/CASH_OUT, GROUP BY is_fraud: % sender emptied, % receiver balance stays 0, % sender balance mismatch, average and median amount (PERCENTILE_APPROX), count at the 10M cap.
`repeat_fraud_destinations`	Fraud only, GROUP BY name_dest HAVING COUNT(*) > 1: receivers that got several frauds (possible mule accounts).

Example, the balance_patterns query:

```
"balance_patterns": """
    SELECT CASE WHEN is_fraud = 1 THEN 'fraud' ELSE 'legit' END AS class,
           COUNT(*) AS transactions,
           ROUND(100 * AVG(orig_emptied), 2) AS pct_origin_emptied,
           ROUND(100 * AVG(CASE WHEN old_balance_dest = 0 AND new_balance_dest = 0 THEN 1 ELSE 0 END), 2)
               AS pct_dest_balance_unchanged_zero,
           ROUND(100 * AVG(CASE WHEN ABS(error_balance_orig) > 0.01 THEN 1 ELSE 0 END), 2)
               AS pct_origin_balance_mismatch,
           ROUND(AVG(amount), 2) AS avg_amount,
           ROUND(PERCENTILE_APPROX(amount, 0.5), 2) AS median_amount,
           SUM(CASE WHEN amount = 10000000 THEN 1 ELSE 0 END) AS at_10m_cap
    FROM tx
    WHERE type IN ('TRANSFER', 'CASH_OUT')
    GROUP BY is_fraud
```

spark_jobs/analytics.py, lines 75–88

spark_jobs/train_model.py

Step 3: train, evaluate and save the fraud model with Spark MLlib · 164 lines

load_training_data(spark)

Reads only the TRANSFER and CASH_OUT partitions (partition pruning), adds the is_transfer feature and a numeric label, and computes a class weight so that fraud rows (1 in 337) carry the same total weight as legitimate rows.

```
def load_training_data(spark):
    df = (
        spark.read.parquet(str(LAKE_DIR))
        .filter(F.col("type").isin("TRANSFER", "CASH_OUT"))
        .withColumn("is_transfer", (F.col("type") == "TRANSFER").cast("int"))
        .withColumn("label", F.col("is_fraud").cast("double"))
    )
    n_fraud = df.filter("is_fraud = 1").count()
    n_legit = df.filter("is_fraud = 0").count()
    # balanced class weights so the minority class isn't ignored
    df = df.withColumn(
        "weight",
        F.when(F.col("is_fraud") == 1, n_legit / n_fraud).otherwise(1.0),
    )
    return df, n_fraud, n_legit
```

spark_jobs/train_model.py, lines 53–67

Model pipelines

Both models are Spark ML Pipelines. `VectorAssembler` packs the 9 feature columns into one vector column, which MLlib models require. Logistic Regression first standardises the features (`StandardScaler`, mean 0 and standard deviation 1), because it is sensitive to scale and amounts reach millions while hour is 0–23. Random Forest does not need scaling, because trees only compare values with thresholds.

```
lr = Pipeline(stages=[
```

```
    assembler,
    StandardScaler(inputCol="features", outputCol="scaled", withMean=True),
    LogisticRegression(featuresCol="scaled", labelCol="label", weightCol="weight", maxIter=100),
])
rf = Pipeline(stages=[
    assembler,
    RandomForestClassifier(labelCol="label", numTrees=60, maxDepth=12, seed=42),
])
```

spark_jobs/train_model.py, lines 108–116

main(): the training flow

1. `randomSplit([0.7, 0.3], seed=42)` splits the data into training and test sets. Both are cached, because they are used several times.
2. Each pipeline is fitted on the training set and timed, then evaluated on the test set.
3. The existing rule is evaluated by using `is_flagged_fraud` directly as the prediction.
4. Metrics, model info and the Random Forest feature importances are published.
5. The Random Forest scores the test set, and `vector_to_array(probability)[1]` extracts the fraud probability. This is a JVM function, so no Python workers are needed.
6. Precision and recall are computed at thresholds 0.1–0.9 (`threshold_curve`).
7. Every test transaction that was flagged or was a missed fraud is published with an outcome label: true positive, false positive or missed fraud.
8. The model is saved to `models/fraud_rf/` and its file names are shortened (`shorten_part_files`, Section 13).

evaluate(name, predictions)

Builds the confusion matrix with one `groupBy("is_fraud", "prediction").count()` (a single distributed aggregation over 831k rows), derives precision, recall and F1, and adds AUC-ROC and AUC-PR from `BinaryClassificationEvaluator` when the model produced raw scores.

```
def evaluate(name, predictions, train_seconds=None):
    counts = {
        (r["is_fraud"], int(r["prediction"])): r["count"]
        for r in predictions.groupBy("is_fraud", "prediction").count().collect()
    }
    tp, fp = counts.get((1, 1), 0), counts.get((0, 1), 0)
    fn, tn = counts.get((1, 0), 0), counts.get((0, 0), 0)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
```

spark_jobs/train_model.py, lines 70–79

7.4 Windows file system support (spark_jobs/winfs)

spark_jobs/winfs/src/bda/WinRawLocalFileSystem.java

Replaces the Hadoop calls that need winutils.exe · 54 lines

Spark writes files through Hadoop's `RawLocalFileSystem`. On Windows, three of its methods call native helpers that Apache does not ship (`winutils.exe`, `hadoop.dll`), so every write fails. This subclass overrides them with plain `java.io.File` operations:

- `setPermission`: does nothing. Unix permissions mean nothing on a single-user Windows laptop.
- `getFileStatus`: builds the file status from java.io.File instead of calling the native stat helper.
- `listStatus`: lists a directory with `File.list()` instead of the native access check. Files deleted between listing and stat (which happens during task commits) are skipped.

```
public class WinRawLocalFileSystem extends RawLocalFileSystem {

    @Override
    public void setPermission(Path p, FsPermission permission) {
    }
```

```
    @Override
    public FileStatus getFileStatus(Path f) throws IOException {
        File local = pathToFile(f);
        if (!local.exists()) {
            throw new FileNotFoundException("File " + f + " does not exist");
        }
        boolean dir = local.isDirectory();
        return new FileStatus(dir ? 0 : local.length(), dir, 1, getDefaultBlockSize(f),
                local.lastModified(), makeQualified(f));
    }

    @Override
    public FileStatus[] listStatus(Path f) throws IOException {
        File local = pathToFile(f);
        if (!local.exists()) {
            throw new FileNotFoundException("File " + f + " does not exist");
        }
        if (!local.isDirectory()) {
            return new FileStatus[] {getFileStatus(f)};
        }
        String[] names = local.list();
        if (names == null) {
            throw new IOException("Could not list " + f);
        }
        List<FileStatus> result = new ArrayList<>();
        for (String name : names) {
            try {
                result.add(getFileStatus(new Path(f, name)));
            } catch (FileNotFoundException ignored) {
                // deleted between list() and stat, e.g. by a concurrent task commit
            }
        }
        return result.toArray(new FileStatus[0]);
    }
}
```

spark_jobs/winfs/src/bda/WinRawLocalFileSystem.java, lines 20–61

spark_jobs/winfs/src/bda/WinLocalFileSystem.java

Checksummed wrapper that Spark expects · 8 lines

Parts of Spark cast the "file" file system to `LocalFileSystem` (the checksummed wrapper), so the raw class is wrapped in it.

```
public class WinLocalFileSystem extends LocalFileSystem {
    public WinLocalFileSystem() {
        super(new WinRawLocalFileSystem());
    }
}
```

spark_jobs/winfs/src/bda/WinLocalFileSystem.java, lines 6–11

spark_jobs/winfs/winfs.jar

The two classes compiled with the JDK (javac --release 8), 2 KB

`common.py` adds the jar to the driver classpath and sets `spark.hadoop.fs.file.impl = bda.WinLocalFileSystem`, only when `os.name == "nt"`. On Linux, macOS and real clusters, Spark's normal file system is used. The jar goes on the driver classpath rather than `spark.jars`, because distributing jars also triggers the chmod that fails on Windows.

7.5 Exploratory analysis notebook

notebooks/eda.ipynb

Exploration that informed the pipeline design, saved with outputs

Runs Spark on the Parquet lake. The sections, in order:

1. Setup: Spark session, load the lake, file sizes, row count.
2. Schema and sample rows.
3. Parquet partitioning: shows PartitionFilters in Spark's physical plan, and CSV vs Parquet size per partition.

4. Transaction types and fraud, with a bar chart.
5. The existing rule (isFlaggedFraud) vs actual fraud.
6. Time of day: volume vs fraud rate chart.
7. Amount distributions on a log scale; the 10M cap.
8. Balance inconsistencies: defines the error_balance features.
9. Repeat fraud destinations.
10. Summary table linking each finding to a modelling decision.

7.6 Backend API (backend/app)

backend/app/main.py

FastAPI application: routes, validation, static files · 43 lines

- `GET /api/source`: tells the dashboard where its data comes from.
- `GET /api/results/{name}`: returns a result set. Names are checked against an allow-list (`RESULT_SETS`), so the URL can't be used to read arbitrary collections or files.
- `GET /api/flagged`: paged flagged transactions. `outcome` is restricted to the three valid values, and `limit` is capped at 500.
- `POST /api/predict`: validates the body with the `Transaction` model, then calls predict.score. If Java is missing, it returns HTTP 503 with a clear message instead of crashing.
- Everything else is served from `frontend/` (`app.mount`), so one server provides both the API and the page.

```
class Transaction(BaseModel):
    type: Literal["TRANSFER", "CASH_OUT"]
    amount: float = Field(gt=0)
    old_balance_orig: float = Field(ge=0)
    new_balance_orig: float = Field(ge=0)
    old_balance_dest: float = Field(ge=0)
    new_balance_dest: float = Field(ge=0)
    hour_of_day: int = Field(ge=0, le=23)


@app.post("/api/predict")
def predict_transaction(tx: Transaction):
    try:
        return predict.score(tx.model_dump())
    except SystemExit as e:  # raised by get_spark() when Java is missing
        raise HTTPException(503, str(e))
```

backend/app/main.py, lines 42–57

backend/app/store.py

Decides where results are read from · 40 lines

`_mongo_db()` tries to connect to MongoDB (1.5 s timeout) and checks that the `overview` collection has data. The answer is cached (`@lru_cache`), so the check runs once per server start. `get()` and `flagged()` then read from MongoDB (sorted and paged in the database) or from the JSON files (filtered and sliced in Python).

```
def _mongo_db():
    try:
        from pymongo import MongoClient

        client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=1500)
        db = client[MONGO_DB]
        if db.overview.count_documents({}) == 0:
            return None
        return db
    except Exception:
        return None
```

backend/app/store.py, lines 13–23

backend/app/predict.py

Live scoring with the saved Spark model · 46 lines

Spark takes seconds to start, so it starts on the first prediction request and is kept in `_state`. A lock makes sure two simultaneous first requests don't start two Spark sessions. The model is loaded through the JVM (`spark._jvm...PipelineModel.load`), because the Python loader starts Python workers, which crash on Windows with Python 3.12+. `score()` builds the 9 features exactly as in training, makes a one-row DataFrame from pandas (through Arrow), runs the model and returns the probability.

```
def _load():
    with _lock:
        if "model" not in _state:
            from pyspark.ml import PipelineModel

            from spark_jobs.common import get_spark

            spark = get_spark("fraud-api")
            # load through the JVM: PipelineModel.load() in Python starts Python
            # workers, which crash on Windows with Python 3.12+
            jmodel = spark._jvm.org.apache.spark.ml.PipelineModel.load(str(MODEL_DIR))
            _state["spark"] = spark
            _state["model"] = PipelineModel._from_java(jmodel)
    return _state["spark"], _state["model"]
```

backend/app/predict.py, lines 15–28

7.7 Dashboard (frontend)

frontend/index.html

Page structure · 122 lines

A header (title, dataset link, pipeline, serving mode, data source) and 7 numbered sections, each answering one question:

Section	Contents	Element IDs filled by app.js
1. Dataset overview	6 KPI cards + data quality note	stats, qualityNote
2. Which types are affected?	Finding sentence, bar chart, table	typeFinding, typeChart, typeTable
3. When does fraud happen?	Finding, volume + fraud-rate chart	hourFinding, hourChart
4. What does fraud look like?	Amount chart, balance table, mule accounts	amountChart, balanceTable, muleTable
5. Model	Comparison table, confusion matrix, feature importance, threshold curve	modelTable, confusion, featureChart, thresholdChart
6. Flagged transactions	Filter buttons, paged table	filters, flaggedTable, pageInfo
7. Check a transaction	Example buttons, 7-field form, result	predictForm, predictResult

Scripts load in a fixed order: Chart.js, `config.js` (mode flag), `forest.js` (browser model), then `app.js`.

frontend/style.css

Styling · 73 lines

A plain, report-like design: system fonts, a white background, one accent colour (dark blue), red for fraud and grey for legitimate. Colours are CSS variables. Below 800 px width, two-column layouts stack and the KPI grid drops to 2 columns, so the page works on phones.

frontend/app.js

All dashboard behaviour · 271 lines

Helpers

`num`, `money` (bn/M formatting), `pct` for number formatting. `esc` escapes HTML, so account names from the data can't inject markup. `table()` renders any table from headers and rows. `api()` is a fetch wrapper that turns HTTP errors into readable messages.

Data layer

The only place that knows where data comes from. Every other function calls `data.source()`, `data.result(name)`, `data.flagged(...)` or `data.predict(tx)`:

```
// Server mode reads the FastAPI backend; the static build (deploy/build_static.py)
// reads the same result files directly and scores transactions in the browser.
let flaggedCache = null;
const data = window.STATIC_SITE
  ? {
      source: async () => "results/*.json (static build)",
      result: (name) => api(`data/${name}.json`),
      async flagged({ outcome, skip, limit }) {
        flaggedCache = flaggedCache || (await api("data/flagged_transactions.json"));
        const rows = outcome ? flaggedCache.filter((r) => r.outcome === outcome) : flaggedCache;
        return { items: rows.slice(skip, skip + limit), total: rows.length };
      },
      predict: (tx) => scoreWithForest(tx),
    }
  : {
      source: async () => (await api("/api/source")).source,
      result: (name) => api(`/api/results/${name}`),
      flagged({ outcome, skip, limit }) {
        const q = new URLSearchParams({ skip, limit });
        if (outcome) q.set("outcome", outcome);
        return api(`/api/flagged?${q}`);
      },
      predict: (tx) => api("/api/predict", { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(tx) }),
    };
```

frontend/app.js, lines 23–46

Render functions

`renderOverview`, `renderTypes`, `renderHourly`, `renderAmounts`, `renderBalances`, `renderMules` and `renderModel` each fill one section. The finding sentences are computed from the data, for example the hour with the highest fraud rate, and the fraud types are the ones with a non-zero fraud count. Rerunning the pipeline on different data would update the text automatically.

Flagged table, form and startup

`flaggedState` holds the current filter and page. The filter buttons and the previous/next buttons update it and call `loadFlagged()`. `fillExample()` fills the form with a fixed normal cash-out or with a real true-positive fraud from the test set. The form's submit handler builds the request, calls `data.predict` and shows the result. `main()` loads everything in parallel and renders the page.

```
async function main() {
  const names = ["overview", "data_quality", "type_summary", "hourly", "amount_bands",
"balance_patterns",
    "repeat_fraud_destinations", "model_metrics", "model_info", "feature_importance", "threshold_curve"];
  const [src, ...results] = await Promise.all([data.source(), ...names.map((n) => data.result(n))]);
  const r = Object.fromEntries(names.map((n, i) => [n, results[i]]));

  $("source").textContent = src;
  if (window.STATIC_SITE) {
    $("serving").textContent = "static site, Random Forest exported from Spark and run in the browser";
    $("predictNote").textContent =
      "Scores a single transaction with the trained Random Forest. Its 60 trees were exported from Spark
MLlib " +
      "and run here in your browser; they give the same probabilities as Spark.";
  }
  renderOverview(r.overview[0], r.data_quality[0]);
  renderTypes(r.type_summary, r.overview[0]);
  renderHourly(r.hourly);
  renderAmounts(r.amount_bands);
  renderBalances(r.balance_patterns);
  renderMules(r.repeat_fraud_destinations);
  renderModel(r.model_metrics, r.model_info[0], r.feature_importance, r.threshold_curve);
```

```
  await loadFlagged();
```

frontend/app.js, lines 271–291

frontend/forest.js

Random Forest scoring in the browser (static mode) · 31 lines

Fetches `data/forest.json` once and reuses it. It builds the 9 features in the same order and with the same formulas as training, routes them through every tree, averages the leaf probabilities, and flags fraud when the average is above 0.5. This is the same rule Spark uses: pick the class with the higher probability.

```
async function scoreWithForest(tx) {
  const forest = await loadForest();
  const errorOrig = tx.new_balance_orig + tx.amount - tx.old_balance_orig;
  const errorDest = tx.old_balance_dest + tx.amount - tx.new_balance_dest;
  const x = [
    tx.type === "TRANSFER" ? 1 : 0, tx.amount, tx.old_balance_orig, tx.new_balance_orig,
    tx.old_balance_dest, tx.new_balance_dest, errorOrig, errorDest, tx.hour_of_day,
  ];

  let total = 0;
  for (const tree of forest.trees) {
    let i = 0;
    while (tree.f[i] !== -1) i = x[tree.f[i]] <= tree.t[i] ? tree.l[i] : tree.r[i];
    total += tree.p[i];
  }
  const p = total / forest.trees.length;
```

frontend/forest.js, lines 13–28

frontend/config.js

Mode flag · 2 lines

Contains `window.STATIC_SITE = false;`. The static build overwrites it with `true`. That one line switches app.js between the API and static files.

frontend/vendor/chart.umd.min.js

Chart.js 4.4.4 (third-party, MIT licence)

Bundled instead of loaded from a CDN, so the dashboard also works offline, for example in a classroom without Wi-Fi.

7.8 Deployment files

deploy/build_static.py

Builds the static site and exports the model · 89 lines

1. `export_forest()` reads the Random Forest's node table directly from the model's Parquet file with pyarrow, without Spark or Java. That file has 14,182 rows, one per node, with tree ID, node ID, split feature, threshold, children and class counts. They become compact arrays per tree.
2. `validate()` scores all 2,424 test transactions that Spark scored, using a Python re-implementation of the forest, and compares them with Spark's stored probabilities. The build stops if any differs by more than 0.0001. The actual maximum difference is 0.00005, which is just Spark's 4-decimal rounding.
3. It copies `frontend/` into `site/` and the result JSON files into `site/data/`, writes `forest.json`, and sets `config.js` to static mode.

```
def fraud_probability(forest, x):
    # same as Spark: each tree contributes its leaf's class distribution; average them
    total = 0.0
    for tree in forest["trees"]:
        i = 0
        while tree["f"][i] != -1:
            i = tree["l"][i] if x[tree["f"][i]] <= tree["t"][i] else tree["r"][i]
        total += tree["p"][i]
    return total / len(forest["trees"])
```

```
def validate(forest):
    rows = json.loads((RESULTS_DIR / "flagged_transactions.json").read_text())
    worst = max(abs(fraud_probability(forest, features_of(r)) - r["fraud_probability"]) for r in rows)
    print(f"Checked {len(rows)} test transactions against Spark: max difference {worst:.6f}")
    if worst > 1e-4:  # Spark's values were stored rounded to 4 decimals
        sys.exit("Exported forest does not match Spark's predictions")
```

deploy/build_static.py, lines 72–88

The result is 60 trees with 14,182 nodes in 271 KB. For each tree: f = feature index (−1 for a leaf), t = threshold, l / r = left and right child node, p = fraud probability at a leaf. The first three nodes of tree 0:

```
{"f":[2,8,5],"t":[327397.37,7.5,55262.41],"l":[1,2,3],"r":[226,105,18],"p":[0,0,0]}
```

Node 0 splits on feature 2 (old_balance_orig) at 327397.37; node 1 on feature 8 (hour_of_day) at 7.5.

deploy/deploy_hf.py

Publishes the static site · 39 lines

Runs the build, finds the logged-in Hugging Face user, creates the Space if it doesn't exist (`space_sdk="static"`), uploads `site/` and deletes files from older builds, then uploads the Space README. Login is done once by the owner with `hf auth login`.

```
api.create_repo(repo_id, repo_type="space", space_sdk="static", exist_ok=True)
api.upload_folder(
    repo_id=repo_id,
    repo_type="space",
    folder_path=build_static.SITE,
    # remove files no longer in the build (keeps the Space's README and .gitattributes)
    delete_patterns=["*.html", "*.js", "*.css", "data/**", "vendor/**"],
    commit_message="Deploy dashboard",
)
```

deploy/deploy_hf.py, lines 27–35

deploy/space_README.md

Description and settings for the Hugging Face Space page

The YAML header tells Hugging Face this is a static Space (`sdk: static`) and sets its title, colours and short description.

Dockerfile

Container image for a server-based deployment · 22 lines

For hosts that run containers (Render, Railway, a VM). It installs Java 17 and the runtime Python packages, runs as a non-root user, and starts run.py on port 7860 with 2 GB for Spark. It was tested on Linux (Python 3.11, Java 17) with the same settings. It isn't used by the live site, because Docker Spaces on Hugging Face need a paid plan.

```
# Serves the dashboard + API. The Spark pipeline is not run in the container:
# its outputs (results/ and models/) are part of the repository.
FROM python:3.11-slim-bookworm

# Java is needed by Spark for the live prediction endpoint; procps provides `ps`,
# which Spark's launch scripts call
RUN apt-get update \
    && apt-get install -y --no-install-recommends openjdk-17-jre-headless procps \
    && rm -rf /var/lib/apt/lists/*

# Hugging Face Spaces runs containers as uid 1000
RUN useradd -m -u 1000 user
USER user
ENV HOME=/home/user \
    PATH=/home/user/.local/bin:$PATH \
    HOST=0.0.0.0 \
    PORT=7860 \
    SPARK_DRIVER_MEMORY=2g
WORKDIR /home/user/app

COPY --chown=user requirements-app.txt .
RUN pip install --no-cache-dir --user -r requirements-app.txt
```

```
COPY --chown=user . .

EXPOSE 7860
CMD ["python", "run.py"]
```

Dockerfile, complete file

`.dockerignore / requirements-app.txt / requirements.txt`

Build and dependency files

- `.dockerignore` keeps the virtual environment, local Java and MongoDB, raw data, the lake, notebooks and docs out of the image.
- `requirements-app.txt`: only what the server needs (pyspark, pandas, numpy, pyarrow, setuptools, pymongo, fastapi, uvicorn).
- `requirements.txt`: includes requirements-app.txt plus kagglehub (download), matplotlib and ipykernel (notebook) and huggingface_hub (deploy).
- `setuptools` is listed because PySpark 3.5's Arrow conversion imports `distutils`, which Python 3.12+ removed. setuptools provides it.

`.gitignore / .gitattributes`

Version control settings

`.gitignore` excludes generated and large items (venv, raw data, lake, site, jdk, mongodb, Hadoop .crc checksum files, logs). `.gitattributes` marks the model files, jar, parquet, docx and pdf as binary, so Git never rewrites their line endings and corrupts them.

7.9 Pipeline outputs committed to the repository

`models/fraud_rf/`

Trained Random Forest pipeline in Spark ML format (347 KB)

Path inside the folder	Contents
`metadata/part-00000`	JSON: pipeline class, Spark version, list of stage IDs
`stages/0_VectorAssembler_*/metadata/`	JSON: input columns (the 9 features) and output column
`stages/1_RandomForestClassifier_*/metadata/`	JSON: parameters (60 trees, depth 12, seed 42), number of features and classes
`stages/1_RandomForestClassifier_*/data/part-00000.parquet`	The node table: one row per node of every tree (14,182 rows)
`stages/1_RandomForestClassifier_*/treesMetadata/part-00000.parquet`	One row per tree: tree parameters and weight (all 1.0)
`_SUCCESS`	Empty marker files written by Spark when a write completes

`results/*.json`

The 13 result sets (also loaded into MongoDB)

File	Rows	Written by	Fields
data_quality	1	ingest	total_rows, rows_with_nulls, negative/zero amounts, duplicate_rows, distinct accounts, rows_written, seconds
overview	1	analytics	transactions, total_amount, fraud_transactions, fraud_amount, fraud_rate_pct, flagged_by_existing_rule, accounts, days

File	Rows	Written by	Fields
type_summary	5	analytics	type, transactions, total/avg amount, fraud_transactions, fraud_rate_pct
hourly	24	analytics	hour_of_day, transactions, fraud_transactions, fraud_rate_pct
daily	31	analytics	day, transactions, fraud_transactions, total_amount
amount_bands	7	analytics	band, legit, fraud
balance_patterns	2	analytics	class, transactions, % emptied, % receiver stays 0, % mismatch, avg/median amount, at_10m_cap
repeat_fraud_destinations	43	analytics	name_dest, fraud_transactions, total_amount, first_step, last_step
model_metrics	3	train_model	model, precision, recall, f1, tp, fp, fn, tn, train_seconds, areaUnderROC, areaUnderPR
model_info	1	train_model	training_rows, test_rows, test_fraud, fraud_weight, features, selected_model
feature_importance	9	train_model	feature, importance
threshold_curve	9	train_model	threshold, precision, recall, alerts
flagged_transactions	2,424	train_model	step, day, hour, type, amount, accounts, 4 balances, fraud_probability, is_fraud, outcome

docs/

Documentation

The project report (Word and PDF), which focuses on findings, results and viva preparation, and this technical documentation.

8. How the model works

8.1 Where the labels come from

Each transaction in PaySim has an isFraud label (1 or 0). The simulator knows which simulated customers are fraudsters, so the labels are ground truth. In a real bank they would come from confirmed cases (chargebacks, complaints, investigations). The model learns from 8,197 labelled frauds. This is supervised learning.

8.2 Features

#	Feature	Formula / meaning	Importance
0	is_transfer	1 if TRANSFER, 0 if CASH_OUT	0.068
1	amount	Transaction amount	0.107
2	old_balance_orig	Sender balance before	0.039
3	new_balance_orig	Sender balance after	0.400
4	old_balance_dest	Receiver balance before	0.038
5	new_balance_dest	Receiver balance after	0.043
6	error_balance_orig	new_balance_orig + amount − old_balance_orig	0.195
7	error_balance_dest	old_balance_dest + amount − new_balance_dest	0.050
8	hour_of_day	step mod 24	0.060

8.3 Decision trees and the forest

A decision tree is a flowchart of yes/no questions. During training, at each node Spark tries candidate splits (a feature and a threshold; maxBins = 32 candidate thresholds per feature) and keeps the one that best separates fraud from legitimate rows, measured by Gini impurity. It repeats this until the tree reaches depth 12 or a node is pure.

A Random Forest trains 60 trees, each on a random bootstrap sample of the rows and with a random subset of features considered at each split, so the trees learn different rules. One real path from tree 0, in plain words:

```
sender balance before <= 327,397 ?           yes
  hour of day <= 7 (night) ?                 yes
    receiver balance after <= 55,262 ?       yes
      sender balance dropped by exactly the amount (error_balance_orig ~ 0) ?   yes
        receiver balance rose by exactly the amount (error_balance_dest ~ 0) ? yes
          sender balance after <= 13.73 ?
              yes -> FRAUD      no -> LEGITIMATE
```

8.4 From votes to a decision

1. Compute the 9 features.
2. Route the transaction through each of the 60 trees to a leaf.
3. Each leaf has class counts from training (e.g. 3 legit, 297 fraud), normalised to a probability (0.99).
4. Average the 60 probabilities to get the fraud probability.
5. If it is above 0.5 the transaction is fraud, otherwise legitimate.

8.5 Evaluation

Model	Precision	Recall	F1	AUC-ROC	AUC-PR	TP	FP	FN
Existing rule	1.000	0.002	0.003	–	–	4	0	2,403
Logistic Regression	0.043	0.909	0.083	0.979	0.549	2,189	48,466	218
Random Forest	**0.993**	**0.998**	**0.995**	**0.999**	**0.998**	**2,401**	**17**	**6**

Test set: 831,437 transactions (2,407 frauds), never seen during training.

9. Data formats

9.1 Parquet lake schema (data/lake/transactions)

Column	Type	Source
step	int	raw
amount	double	raw
name_orig	string	raw (nameOrig)
old_balance_orig	double	raw (oldbalanceOrg)
new_balance_orig	double	raw (newbalanceOrig)
name_dest	string	raw (nameDest)
old_balance_dest	double	raw (oldbalanceDest)
new_balance_dest	double	raw (newbalanceDest)
is_fraud	int	raw (isFraud)
is_flagged_fraud	int	raw (isFlaggedFraud)
day	int	derived
hour_of_day	int	derived
dest_is_merchant	int	derived
error_balance_orig	double	derived
error_balance_dest	double	derived
orig_emptied	int	derived
type	string	partition column (folder name type=...)

9.2 MongoDB

Database `bank_analytics`, one collection per result set (13 collections, same names as the results files). Each pipeline run replaces a collection's contents. Example document from `hourly`:

```
{
  "hour_of_day": 4,
  "transactions": 1241,
  "fraud_transactions": 274,
  "fraud_rate_pct": 22.079
}
```

10. API reference (server mode)

Method and path	Parameters	Returns
GET /api/source	–	{ "source": "MongoDB" } or "results/*.json"
GET /api/results/{name}	name: one of the 12 result sets	Array of documents
GET /api/flagged	outcome (optional), skip ≥ 0, limit 1–500	{ items: [...], total: n }
POST /api/predict	JSON body, see below	{ fraud_probability, is_fraud, error_balance_orig, error_balance_dest }

Example request and response:

```
POST /api/predict
{"type": "CASH_OUT", "amount": 2658595.89, "hour_of_day": 2,
 "old_balance_orig": 2658595.89, "new_balance_orig": 0,
 "old_balance_dest": 286931.14, "new_balance_dest": 2945527.03}

200 OK
{"fraud_probability": 1.0, "is_fraud": true,
 "error_balance_orig": 0.0, "error_balance_dest": 0.0}
```

Invalid input (for example type PAYMENT or a negative amount) returns 422 with details. If Java is not available, predict returns 503.

11. Deployment

Live site: https://jayquery-007-bank-transaction-analytics.static.hf.space

11.1 Why a static site

The first plan was a Docker Space running FastAPI and Spark. When deploying, Hugging Face returned "402 Payment Required": Docker Spaces on free hardware now need a paid PRO plan, and the free tiers of other hosts typically offer about 512 MB of memory, which isn't enough for Spark. Only the prediction form actually needs computation, so the trained model was exported to JSON and evaluated in the browser. Everything else is pre-computed results. This is free and permanent, and predictions are instant.

11.2 How to redeploy

```
hf auth login              # once per machine (browser sign-in)
python deploy/deploy_hf.py  # build site/ and upload it
```

11.3 Server deployment alternative

For a full server deployment (FastAPI + Spark + MongoDB), build the Dockerfile on any container host with about 2 GB of memory, and optionally set MONGO_URI to a hosted MongoDB such as MongoDB Atlas.

12. Running and reproducing

Goal	Command	Needs
Set up	python -m venv venv; venv\Scripts\activate; pip install -r requirements.txt	Python 3.10+
Open the dashboard	python run.py → http://localhost:8000	nothing else
Dashboard with MongoDB (Windows)	.\start.ps1	portable MongoDB in mongodb/
Rebuild everything from raw data	python run_pipeline.py	Java 17, Kaggle token (or manual CSV)
Run one step	python -m spark_jobs.ingest (or analytics, train_model)	Java 17
Explore	open notebooks/eda.ipynb	Java 17, the lake
Deploy	python deploy/deploy_hf.py	hf auth login

Pipeline step	Runtime (16 GB laptop)	Output
download	~20 s	data/raw/paysim.csv
ingest	20–40 s	6,362,604 rows in the Parquet lake
analytics	25–40 s	7 result sets
train_model	~2.5 min	metrics, model, flagged transactions

13. Problems encountered and how they were solved

Problem	Cause	Solution
Spark could not write Parquet or save the model on Windows	Hadoop calls winutils.exe / hadoop.dll, which Apache doesn't ship for Windows	Wrote WinRawLocalFileSystem / WinLocalFileSystem (Section 7.4) instead of downloading unofficial binaries
Loading the jar with spark.jars failed	Distributing jars also triggers a chmod	Put the jar on spark.driver.extraClassPath
PySpark Python workers crashed (WinError 10038)	PySpark worker sockets fail on Windows with Python 3.12+; Spark 4.0 had the same problem	No Python UDFs/RDDs; model loaded through the JVM; Arrow for conversions
toPandas() failed: no module named distutils	Python 3.12 removed distutils; PySpark 3.5 imports it	Added setuptools, which provides distutils
pip could not install numpy 1.26 on Python 3.13	No prebuilt wheel; building needs a C compiler	Moved to numpy 2.x and pandas 2.2+, which ship Python 3.13 wheels
git clone failed: filename too long	Spark's part-file names pushed paths past Windows' 260-character limit	Rename model part files after saving (shorten_part_files)
MongoDB and Java installers failed (MSI error 1618)	Windows Installer lock on the development laptop	Used portable ZIP versions of MongoDB and the JDK, which need no installation
Hugging Face Docker Space refused (402)	Docker Spaces now need a paid plan	Static Space with the model exported to JSON (Section 11)

14. Testing and verification

Check	Result
Fresh clone + new virtual environment + pip install	Installs cleanly
Full pipeline from raw data in a fresh clone	Completes; reproduces the same metrics (same 17 false positives and 6 missed frauds)
Dashboard in server mode with MongoDB	All sections load; source shows MongoDB
Dashboard in server mode without MongoDB	Falls back to results/*.json
Live prediction (server mode)	Real test-set fraud scores 1.000; normal cash-out scores 0.000
Invalid prediction input	Rejected with HTTP 422
Container setup on Linux (Python 3.11, Java 17, Dockerfile environment)	All endpoints work; Spark uses about 650 MB
Exported forest (Python) vs Spark, 2,424 test transactions	Maximum difference 0.00005 (Spark's 4-decimal rounding)
Browser forest (JavaScript) vs Spark, 2,424 test transactions	Same probabilities; 0 decision disagreements
Live site	All charts, tables, filters and the prediction form work; no browser errors

15. Limitations and future work

- **Simulated data:** PaySim records balances for fraud in a distinctive way, which the balance-error features pick up. That is why the scores are near-perfect, and real data would score lower.
- **No customer history:** senders almost never repeat, so behaviour-over-time features aren't possible.
- **Batch only:** the next step would be Kafka + Spark Structured Streaming, scoring transactions as they arrive.
- **Local mode:** Spark ran on one machine. The code needs no changes to run on a cluster with HDFS or cloud storage.
- **More models:** Gradient-Boosted Trees, unsupervised anomaly detection for new fraud patterns, and graph analysis (Neo4j) for mule-account rings.

Glossary

Term	Meaning
Spark (local[*])	Distributed processing engine; local[*] runs it on all cores of one machine
DataFrame	Spark's distributed table, processed in partitions
Partition pruning	Skipping data folders that a query's filter rules out
Parquet	Columnar, compressed file format that stores its schema
Serving layer	Database holding ready-to-display results (here MongoDB)
MLlib Pipeline	Chain of feature and model stages, saved and loaded as one unit
Random Forest	Many decision trees, each trained on a random sample; their votes are averaged
Precision / Recall	Share of flagged transactions that are fraud / share of frauds that are flagged
AUC-PR	Area under the precision-recall curve; suited to rare classes
Class weight	Giving rare-class rows more weight in training
Static site	Website made only of files, with no server-side code